Laboratorium nr 7 – generowanie języka naturalnego cz. 2

Zadanie na 3.0

Wczytujemy wszystkie dzieła Szekspira z:

```
import tensorflow as tf

shakespeare_url = "https://homl.info/shakespeare" # shortcut URL
filepath = tf.keras.utils.get_file("shakespeare.txt", shakespeare_url)
with open(filepath) as f:
   shakespeare_text = f.read()
```

Wyświetlamy kilka pierwszych wierszy:

```
print(shakespeare_text[:100])
```

W kolejnym kroku kodujemy nasz tekst w postaci znaków (jako małe litery). Każdy znak zostaje przypisany do liczby całkowitej od 2. `TextVectorization` rezerwuje 0 dla tokenów uzupełniających (sztuczne symbole dodawane do sekwencji tekstowych, żeby wszystkie miały taką samą długość), a 1 dla nieznanych znaków.

Nie będziemy potrzebować tych tokenów o wartościach 0 i 1.

Docelowo będziemy mieć 39 różnych znaków – tyle co liter w alfabecie.

```
text_vec_layer = tf.keras.layers.TextVectorization(
   split="character",
   standardize="lower"
)
text_vec_layer.adapt([shakespeare_text])
encoded = text_vec_layer([shakespeare_text])[0]

encoded -= 2 # drop tokens 0 (pad) and 1 (unknown), which we will not use
n_tokens = text_vec_layer.vocabulary_size() - 2 # number of distinct chars
   = 39
dataset_size = len(encoded) # total number of chars = 1,115,394
```

Następnie zakodujemy zbiór danych w postaci sekwencji wymaganych przez sieci rekurencyjne.

Przykład zmienna wejściowa: `To be or not to b`

Zmienna wyjściowa: `o be or not to be`

Zmienne wyjściowe są ogólnie przesunięte o jeden takt w przyszłość.

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32):
   ds = tf.data.Dataset.from_tensor_slices(sequence)
   ds = ds.window(length + 1, shift=1, drop_remainder=True)
   ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
   if shuffle:
      ds = ds.shuffle(100_000, seed=seed)
```

```
    ds = ds.batch(batch_size)
    return ds.map(
        lambda window: (window[:, :-1], window[:, 1:])
    ).prefetch(1)
```

Funkcja ta przyjmuje sekwencje na wejściu (tj. zakodowany tekst) i tworzy zestaw danych zawierający ramki pożądanej długości. Następnie zwiększa długość o 1, gdyż dla zmiennej docelowej potrzebujemy następnego znaku. Następnie tasuje ramki (nieobowiązkowo), tworzy ich grupy, dzieli na pary danych wejściowych/danych wyjściowych, a także aktywuje wstępne ładowanie.

Przykładowe wywołanie:

```
list(to_dataset(text_vec_layer(["To be"])[0], length=4))
```

Widzimy przesunięcie w tej ramce.

Teraz dzielimy zbiór danych na uczący – 90%, walidacyjny 5% oraz testowy 5%. Wyznaczamy długość ramki 100.

```
length = 100
tf.random.set_seed(42)
train_set = to_dataset(encoded[:1_000_000], length=length, shuffle=True,
    seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)
test_set = to_dataset(encoded[1_060_000:], length=length)
```

Teraz budujemy nasz model. Nasz zestaw danych jest dość duży, więc potrzebujemy architektury bardziej zaawansowanej niż kilka rekurencyjnych neuronów. Zbudujemy sieć z jedną warstwą GRU zawierającą 128 jednostek.

```
tf.random.set_seed(42) # extra code  ensures reproducibility on CPU
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
    tf.keras.layers.GRU(128, return_sequences=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="nadam",
    metrics=["accuracy"]
)
model_ckpt = tf.keras.callbacks.ModelCheckpoint(
    "my_shakespeare_model",
    monitor="val_accuracy",
    save_best_only=True
)
history = model.fit(
    train_set,
    validation_data=valid_set,
    epochs=10,
    callbacks=[model_ckpt]
)
```

Nasz model składa się z następujących warstw:

- **Warstwa embedding** – kodowanie identyfikatorów znaków. Liczba wymiarów wejściowych jest taka sama jak liczba różnych identyfikatorów znaków, natomiast liczba wymiarów wyjściowych to modyfikowalny parametr – na razie wyznaczamy jego wartość na 16. Dane wejściowe to tensory dwuwymiarowe $[rozmiargrupy, dugoramki]$, natomiast dane wychodzące z tej warstwy to tensor trzywymiarowy $[rozmiargrupy, dugoramki, rozmiarwektorawaciwociowego$
- **Warstwa wyjściowa Dense** – musi zawierać 39 jednostek, bo mamy 39 różnych znaków w tekście. Suma wartości z tych 39 jednostek da wartość równą 1, a poszczególna jednostka zawiera prawdopodobieństwo wybrania akurat tego znaku (softmax).

Kompilujemy model z funkcją straty `sparse_categorical_crossentropy` i optymalizatorem Nadam i trenujemy kilka epok, zapisując najlepszy model w kontekście dokładności na zbiorze walidacyjnym.

Spróbuj nauczyć model na Google Colab z obsługą GPU – powinno to zająć około 2 godzin. Jeśli posiadasz komputer z kartą graficzną obsługującą technologię CUDA możesz na niej dokonać obliczeń.

```
shakespeare_model = tf.keras.Sequential([
   text_vec_layer,
   tf.keras.layers.Lambda(lambda X: X - 2), # no <PAD> or <UNK> tokens
   model
])
```

Model ten nie zajmuje się wstępnym przetwarzaniem danych tekstowych, dlatego umieśćmy tę możliwość w końcowym modelu zawierającym na początku warstwę `TextVectorization`, a także warstwę `Lambda` odejmującą od identyfikatorów 2 znaki (bo nie korzystamy z uzupełniania i nieznanych znaków). Jest to swego rodzaju adapter pomiędzy surowym tekstem, a wytrenowanym modelem.

Końcowe przewidywanie wykonujemy w następujący sposób:

```
y_proba = shakespeare_model.predict(["To be or not to b"])[0, -1]
y_pred = tf.argmax(y_proba) # choose the most probable character ID
text_vec_layer.get_vocabulary()[y_pred + 2]
```

Spróbuj tym modelem dokonać predykcji całego słowa wywołując odpowiednią liczbę razy predykcję pojedynczego znaku.

Na TEAMS prześlij plik `.zip`, który będzie miał nazwę:

`LAB7_Nazwisko_Imie_Nazwisko_Imie.zip`

Archiwum powinno zawierać kod źródłowy w języku Python i krótkie podsumowanie eksperymentu w pliku `readme`. Zadanie na 3.0 nie wymaga odpowiedzi ustnej ze sprawozdania.

Zadanie na 4.0

Aby wygenerować nowy tekst za pomocą modelu char-RNN moglibyśmy mu dostarczyć jakiś tekst, sprawić aby model przewidział najbardziej prawdopodobną następną literę i dodać ją na koniec tekstu. Jest to tak zwane dekodowanie zachłanne (ang. *greedy decoding*). To spowoduje jednak, że ciągle będą generowane te same słowa. Zamiast tego losujmy te znaki z pewnym oszacowanym prawdopodobieństwem.

Funkcja `categorical()` losuje indeksy klas na podstawie logarytmów prawdopodobieństwa (logitów) przynależności do klasy. Na przykład:

```
log_probas = tf.math.log([[0.5, 0.4, 0.1]]) # probas = 50%, 40%, and 10%
tf.random.set_seed(42)
print(tf.random.categorical(log_probas, num_samples=8)) # draw 8 samples
```

Wylosować może np.:

```
tf.Tensor([[0 1 0 2 1 0 0 1]], shape=(1, 8), dtype=int64)
```

Możemy uzyskać większą kontrolę nad zróżnicowaniem przez dzielenie logitów przez wartość zwaną temperaturą, którą możemy dowolnie modyfikować. Temperatura zbliżona do zera będzie faworyzowała znaki o dużym prawdopodobieństwie, natomiast bardzo duża temperatura wyrównuje szanse wszystkich znaków.

- Niższe temperatury – zależy nam na ścisłym i precyzyjnym tekście.
- Wyższe temperatury – twórczy tekst.

Realizuje to funkcja `next_char` i pomocnicza funkcja `extend_text`, która będzie wywoływać tę funkcję i dołączać do tekstu:

```
def next_char(text, temperature=1):
    y_proba = shakespeare_model.predict([text])[0, -1:]
    rescaled_logits = tf.math.log(y_proba) / temperature
    char_id = tf.random.categorical(rescaled_logits, num_samples=1)[0, 0]
    return text_vec_layer.get_vocabulary()[char_id + 2]
```

```
def extend_text(text, n_chars=50, temperature=1):
    for _ in range(n_chars):
        text += next_char(text, temperature)
    return text
```

Wygenerujemy tekst za pomocą nowo przygotowanych metod.

```
tf.random.set_seed(42) # extra code  ensures reproducibility on CPU
print(extend_text("To be or not to be", temperature=0.01))
print(extend_text("To be or not to be", temperature=1))
print(extend_text("To be or not to be", temperature=100))
```

Nasze wyniki niekoniecznie mogą nas zachwycić. Często wprowadza się dodatkowe techniki jak generowanie k najbardziej prawdopodobnych znaków lub generuje się znaki, których prawdopodobieństwo przekracza jakąś wartość progową (tzw. próbkowanie jądra). Być może potrzebna byłaby bardziej skomplikowana architektura sieci (więcej warstw GRU i więcej neuronów w poszczególnych warstwach; można też zwiększyć ramkę).

Na TEAMS prześlij plik `.zip`, który będzie miał nazwę:

`LAB7_Nazwisko_Imie_Nazwisko_Imie.zip`

Archiwum powinno zawierać pliki źródłowe w języku Python i krótkie sprawozdanie w formacie `.pdf` podsumowujące eksperyment.

Prześlij `.zip` w terminie i podejdź do odpowiedzi ustnej na najbliższych laboratoriach.

Zadanie na 5.0

W tym podpunkcie przejdziemy z bezstanowych sieci rekurencyjnych na sieci stanowe.

W sieciach bezstanowych w każdej iteracji stan ukryty wypełniony jest zerami, po czym w każdym takcie wartości te są aktualizowane, a po ostatnim takcie zostają wyzerowane, gdyż przestają być potrzebne.

W sieciach stanowych każemy zachować sieci stan końcowy po przetworzeniu grupy danych i wykorzystujemy go jako stan początkowy do przetwarzania kolejnej grupy. W ten sposób model mógłby rozpoznawać długoterminowe wzorce nawet mimo propagacji wstecznej przez krótkie sekwencje.

Musimy zmienić podejście do zbioru danych. Każda sekwencja wejściowa w grupie rozpoczyna się tam, gdzie zakończyła się sekwencja w poprzedniej grupie. Potrzebujemy więc sekwencyjne, nienakładające się sekwencje wejściowe (bez tasowania i nakładania jak w sieciach bezstanowych).

Nie skorzystamy też z funkcji `batch(32)`, bo pierwsza grupa zawierałaby ramki 1 i 32, a druga 33 i 64. Więc jeśli rozpatrzymy pierwsze ramki w każdej grupie, czyli ramki 1 i 33, to widać, że nie następują one bezpośrednio po sobie. Najprostsza strategia to wykorzystanie jednoelementowej grupy.

```
def to_dataset_for_stateful_rnn(sequence, length):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=length, drop_remainder=True)
    ds = ds.flat_map(lambda window: window.batch(length + 1)).batch(1)
    return ds.map(
        lambda window: (window[:, :-1], window[:, 1:])
    ).prefetch(1)

stateful_train_set = to_dataset_for_stateful_rnn(encoded[:1_000_000],
    length)
stateful_valid_set = to_dataset_for_stateful_rnn(
    encoded[1_000_000:1_060_000],
    length
)
```

```
stateful_test_set = to_dataset_for_stateful_rnn(
   encoded[1_060_000:],
   length
)
```

Teraz zbudujemy stanowy model sieci rekurencyjnej.

```
tf.random.set_seed(42) # extra code  ensures reproducibility on CPU
model = tf.keras.Sequential([
   tf.keras.layers.Embedding(
      input_dim=n_tokens,
      output_dim=16,
      batch_input_shape=[1, None]
   ),
   tf.keras.layers.GRU(128, return_sequences=True, stateful=True),
   tf.keras.layers.Dense(n_tokens, activation="softmax")
])

class ResetStatesCallback(tf.keras.callbacks.Callback):
   def on_epoch_begin(self, epoch, logs):
      self.model.reset_states()

# extra code  use a different directory to save the checkpoints
model_ckpt = tf.keras.callbacks.ModelCheckpoint(
   "my_stateful_shakespeare_model",
   monitor="val_accuracy",
   save_best_only=True
)

model.compile(
   loss="sparse_categorical_crossentropy",
   optimizer="nadam",
   metrics=["accuracy"]
)
history = model.fit(
   stateful_train_set,
   validation_data=stateful_valid_set,
   epochs=10,
   callbacks=[ResetStatesCallback(), model_ckpt]
)

shakespeare_model = tf.keras.Sequential([
   text_vec_layer,
   tf.keras.layers.Lambda(lambda X: X - 2), # no <PAD> or <UNK> tokens
   model
])

tf.random.set_seed(42)
```

```
print(extend_text("to be or not to be", temperature=0.01))
```

W ramach tego zadania oceń, jak ten model radzi sobie w generowaniu nowego tekstu analogicznie jak realizowano to w zadaniu na 4.0.

Na TEAMS prześlij plik `.zip`, który będzie miał nazwę:

`LAB7_Nazwisko_Imie_Nazwisko_Imie.zip`

Archiwum powinno zawierać pliki źródłowe w języku Python i krótkie sprawozdanie w formacie `.pdf` podsumowujące eksperyment.

Prześlij `.zip` w terminie i podejdź do odpowiedzi ustnej na najbliższych laboratoriach.